

UNIT - 6

* Syntax directed translation

→ In syntax directed translation, along with the grammar we associate some informal notation and these notations are called as semantic rules.

so we can say that

$$\boxed{\text{Grammar} + \text{Semantic rule} = \text{SPT}}$$

→ In syntax directed translation, every non-terminal can get one (or) more than one attribute.

→ The value of these attributes is evaluated by the semantic rules associated with the production rule.

Applications

- (1) Executing Arithmetic Expression
- (2) Conversion from infix to postfix
- (3) Conversion from Binary to decimal
- (4) Generating intermediate code
- (5) type checking
- (6) storing type info into symbol table.

Ex 1:-

Grammar

Rules.

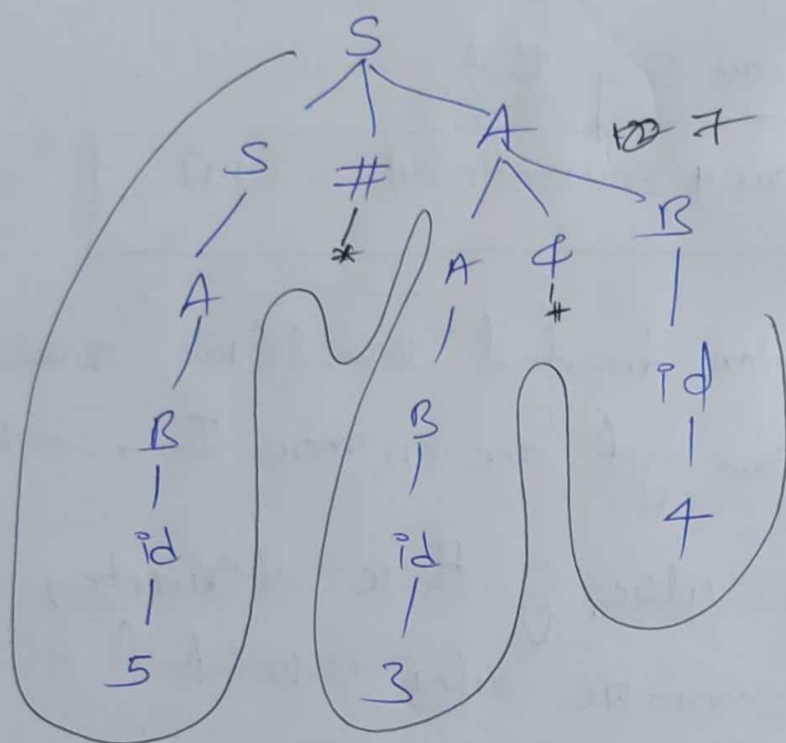
$S \rightarrow S \# A / A \{ S.val = S.val * A.val; \}$

$A \rightarrow A \phi B / B \{ A.val = A.val + B.val; \}$

$B \rightarrow id \{ B.val = id.val; \}$

given string $5 \# 3 \phi 4$?

Sol:-



$$= 5 * (3 + 4) = 5 * 7 = 35.$$

Q2: How to parse SDT Top Down & Bottom up parsing. for given Grammar.

→ The Grammar is

$$S \rightarrow AS \{ \text{printf}(1) \} 1$$

$$S \rightarrow AB \{ \text{printf}(2) \} 2$$

$$A \rightarrow a \{ \text{printf}(3) \} 3$$

$$B \rightarrow bc \{ \text{p.f}(4) \} 4$$

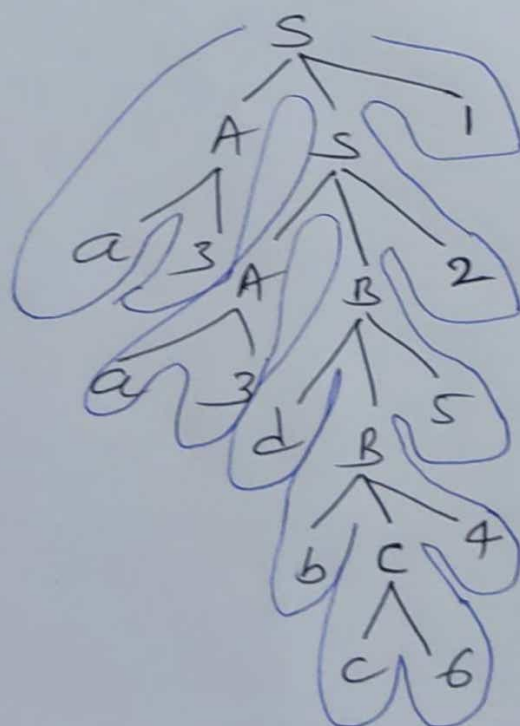
$$B \rightarrow dB \{ \text{p.f}(5) \} 5$$

$$C \rightarrow c \{ \text{p.f}(6) \} 6$$

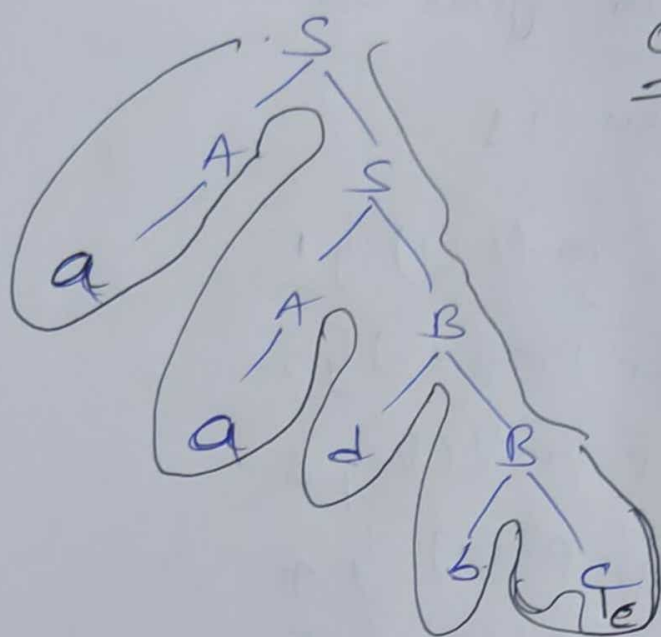
If input = aadbcb

→ Using Top down O/P is

O/P: 3364521

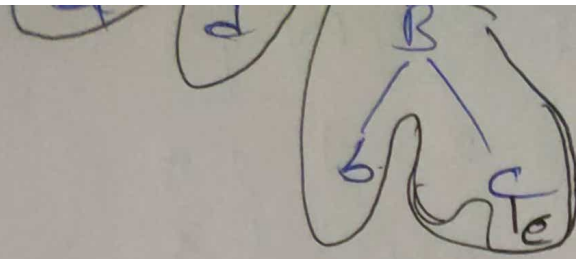


→ Using Bottomup @ that o/p is ?



O/P = 3364521

How to evaluate Arithmetic Expression using
SDT



How to evaluate Arithmetic Expression using
SOT

eg:-

$$E \rightarrow E * T \{ E.val = E.val * T.val \}$$

$$| T \{ E.val = T.val \}$$

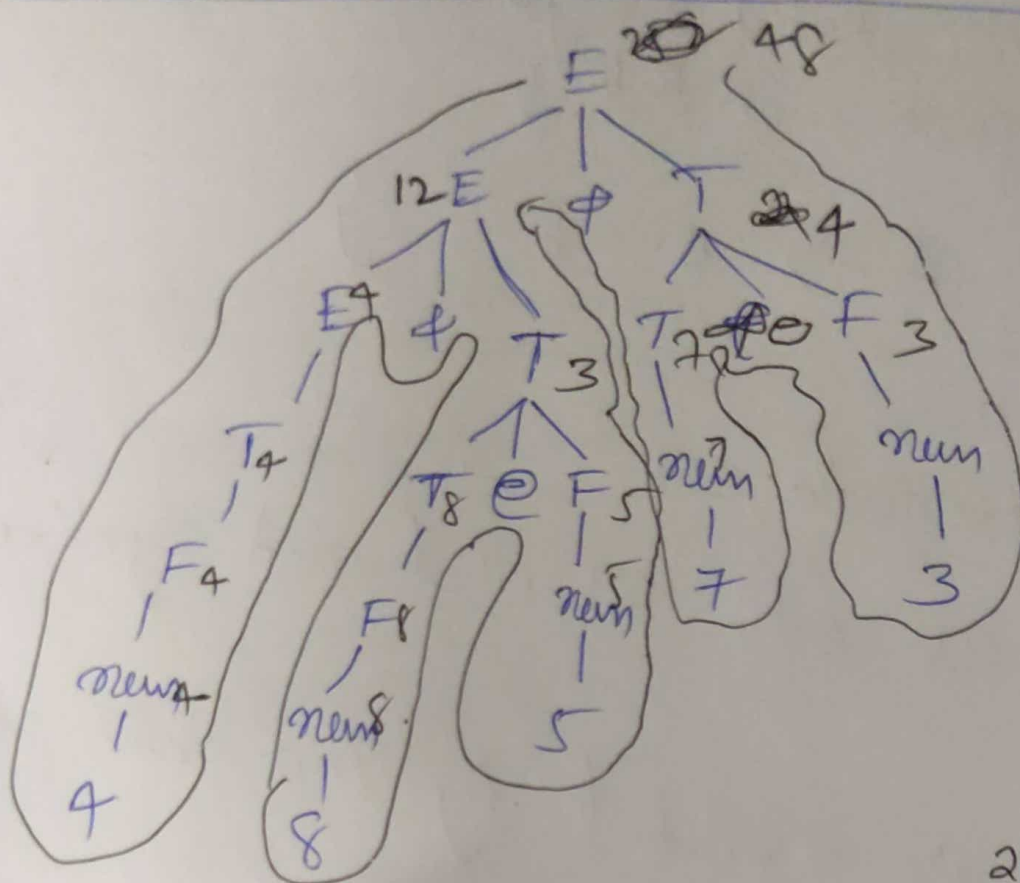
$$T \rightarrow T @ F \{ T.val = T.val - F.val \}$$

$$| F \{ F.val = F.val \}$$

$$F \rightarrow num \{ F.val = num \}$$

$$Exp: \quad 4 * 8 @ 5 * 7 @ 3$$

Sol-



2171 ~ 1

— 252

$$(8-5) = 3$$

Types of SDT

→ It has two types (1) S-attributed SDT
(2) L-attributed SDT.

(1) S-attributed SDT:-

→ It is Based on synthesized attribute

→ We Bottom up parsing.

→ Semantic Rules always written at right most position in R.H.S

eg: $A \rightarrow BC \{ \dots \}$

(2) L-Attribute SDT:-

→ We will use both inherited and synthesized attribute.

→ Attributes are evaluated by Top down parsing

→ Semantic Rules any where in R.H.S

eg: $A \rightarrow \{ \} BC$

$\quad \quad \quad \{ D \{ \} E$

$\quad \quad \quad \{ F \{ \} G \}$

Attributes are 2 types (1) synthesized
(2) Inherited Attribute.

① Synthesized Attribute:-

→ An attribute is said to be synthesized attribute if its parse tree node value is determined by the attribute value at child nodes.

eg:- $A \rightarrow BC \quad \{ A.s = B.s \}$

② Inherited Attribute:-

→ An Attribute is said to be inherited attribute if its parse tree node value is determined by the attribute value at parent (or) sibling node.

eg:- $A \rightarrow BCD \quad \{ \begin{array}{l} C.i = A.i, \\ C.i = B.i, \\ C.i = D.i \text{ (if is not valid)} \end{array} \}$

Gate-2020:-

$A \rightarrow PQ$ and $A \rightarrow XY$

Rule 1: $\{ P.i = A.i + 2, Q.i = P.i + A.i \text{ and } A.s = P.s + Q.s \}$

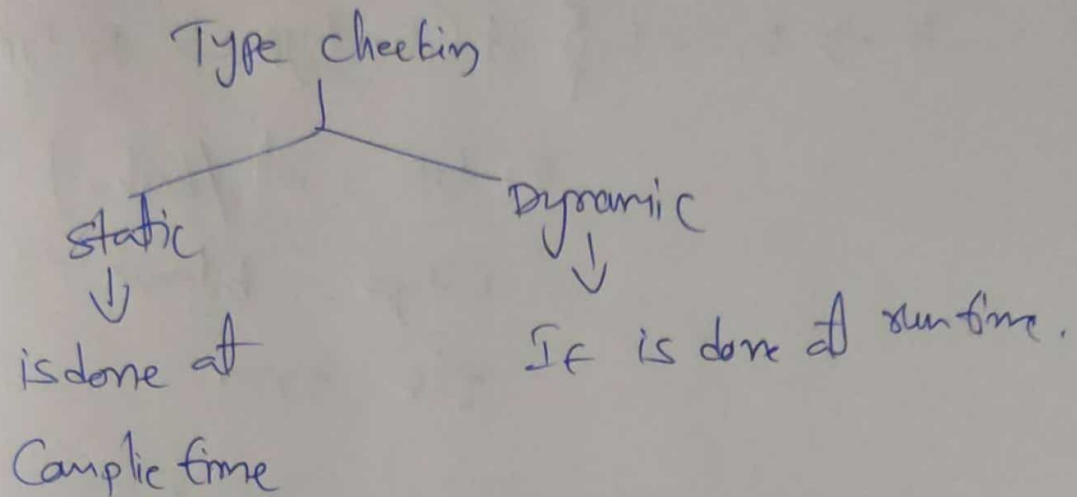
Rule 2: $\{ \textcircled{X} X.i = A.i + Y.s \text{ and } Y.i = X.s + A.i \}$

which of the following is true,

- (a) Both Rules are L-Attributed
- ☒ (b) only R_1 has L-attributed
- (c) only R_2 has L-attributed
- (d) none

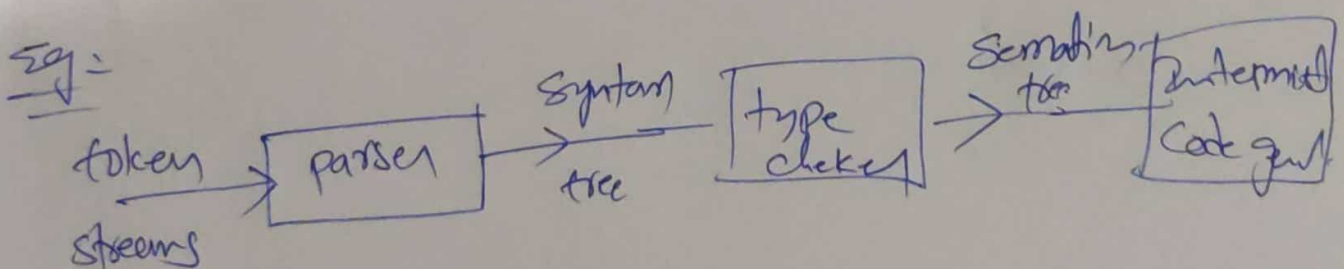
Type checking

→ Type checker is a module of a compiler devoted to type checking tasks.



The design of type checker depends on

- Syntactic structure of language constructs
- The type expression of language.
- The rules of assigning types to construct



Exg:

$E \rightarrow E_1 + E_2 \{ \text{if } (E_1.type == E_2.type) \ \&\& \\ E.type = int \} \text{ then } E.type = int \\ \text{else error } \}$

$E \rightarrow E_1 == E_2 \{ \text{if } (E_1.type == E_2.type) \ \&\& \\ E_1.type = int / boolean \} \text{ then } \\ E.type = boolean \\ \text{else error.}$

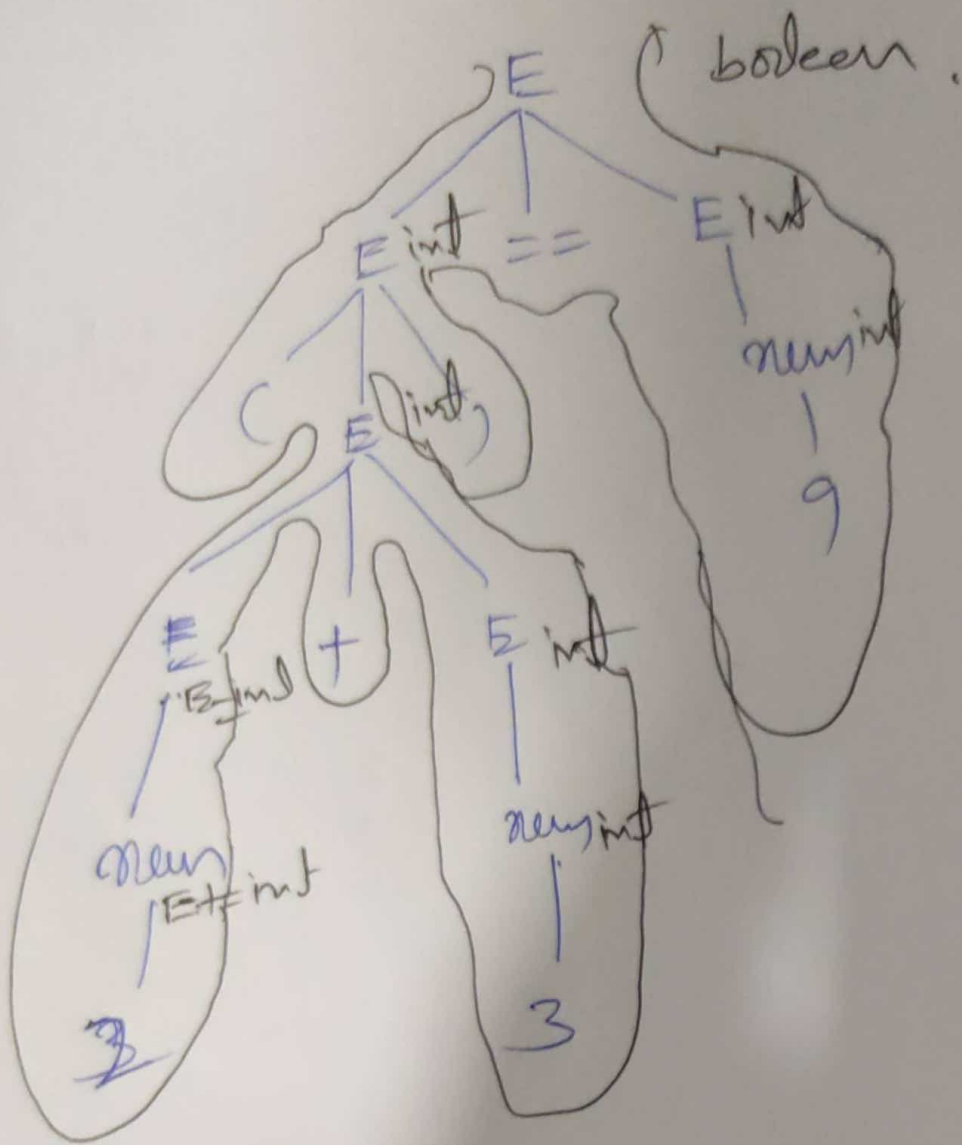
$E \rightarrow (E) \{ E.type = E_1.type \}$

$E \rightarrow \text{sum} \{ E.type = int \}$

$\quad \quad \quad / \text{ true } \{ E.type = boolean \}$

$\quad \quad \quad / \text{ False } \{ E.type = boolean \}$

$\rightarrow$ check for $(2+3) == 9$ it is meaningful
or not.



Intermediate Code Generation

- Machine Independent
- Abstract Syntax tree
- Direct Acyclic graph
- postfix
- 3-address Code.

① Machine Independent:

- Machine independent optimization attempt to improve the intermediate code to get a better target code.
- The part of the code which is transformed here does not involve any absolute memory location
(a) any ~~po~~ CPU register.

② postfix Notation: - The ordinary way of writing the sum of a and b is with operator in the middle $a+b$

- The postfix notation for the same expression places the operator at the right end as $ab+$.

Example: $(a-b) * (c+d) + (a-b)$ is: $ab-cd+ab-+$.

③ Three-Address Code:-

- A statement involving no more than three references is known as a three address statement.
- A sequence of three address statements is known as a three address code.
- Example:- The three address code for the expression

$$a + b * c + d:$$

$$T_1 = b * c$$

$$T_2 = a + T_1$$

$$T_3 = T_2 + d$$

T_1, T_2, T_3 are temporary variables.

④ Syntax tree:-

- Syntax tree is nothing more than condensed form of a parse tree. The operator and keyword nodes of the parse tree are moved to their parents and chain of single productions is replaced by single line in syntax tree the internal nodes are operators and child nodes are operands.

Example:- $a = (a + b * c) / (a - b * c)$

